

Cryptographic Hashing: How It Works

Contents

Notation (list of symbols)	3
1 What a cryptographic hash is	5
1.1 What makes it <i>cryptographic</i>	5
1.2 Symbols	6
2 The Merkle–Damgård construction	7
2.1 The compression function	7
2.2 Padding	7
2.3 Chaining	7
2.4 Why it preserves collision resistance	7
2.5 A built-in weakness: length extension	8
2.6 Symbols	8
3 Popular hash algorithms	9
3.1 The Merkle–Damgård line	9
3.2 Beyond Merkle–Damgård	9
3.3 Symbols	10
4 Attacks and resistance	11
4.1 Brute-force preimage: $\approx 2^n$	11
4.2 Collisions and the birthday bound: $\approx 2^{n/2}$	11
4.3 What this means for digest size	12
4.4 Real-world breaks	12
4.5 Symbols	12
5 Length-extension attacks	14
5.1 What it is	14
5.2 Why it is harmful	14
5.3 A real attack: forged API signatures	14
5.4 The fix	15
5.5 Symbols	15
6 HMAC: keyed authentication	16
6.1 What a MAC is	16
6.2 Why the obvious constructions fail	16
6.3 The construction	17
6.4 Why it works	17
6.5 In practice	18
6.6 Symbols	18
7 Key derivation functions (KDFs)	19
7.1 The general shape	19
7.2 High-entropy material: HKDF (extract-then-expand)	19
7.3 Low-entropy material: password-based KDFs (deliberately slow)	19
7.4 Symbols	20

8	Storing passwords	21
8.1	Why raw $H(\text{password})$ fails	21
8.2	Salting: kill precomputation and duplicates	21
8.3	Slow it down with a password KDF	22
8.4	Symbols	22
9	Merkle trees	23
9.1	Construction	23
9.2	Proving membership	23
9.3	A tiny example	23
9.4	The same proof at real scale	24
9.5	Where they are used	24
9.6	Symbols	24
10	Inside SHA-2 (SHA-256)	25
10.1	The compression function: a block cipher plus feed-forward	25
10.2	The message schedule	25
10.3	The round function	25
10.4	Padding: filling the last block	26
10.5	Putting it together: hashing a message	27
10.6	Symbols	27
11	Inside SHA-3 (the sponge)	29
11.1	State, rate, and capacity	29
11.2	Padding to the rate	29
11.3	Absorb, then squeeze	29
11.4	The permutation P (Keccak- f)	30
11.5	Why industry still leans on SHA-2	32
11.6	Symbols	33
12	Inside BLAKE3	34
12.1	Chunks as the leaves of a tree	34
12.2	Short blocks and lopsided trees	34
12.3	Inside a chunk: chaining the blocks	35
12.4	The compression function f	35
12.5	Extendable output	37
12.6	BLAKE3 vs. SHA-3	37
12.7	Symbols	38

Notation (list of symbols)

H	a (cryptographic) hash function, $\{0,1\}^* \rightarrow \{0,1\}^n$.
m	the message / input; an arbitrary-length bit string, $m \in \{0,1\}^*$.
h	the digest (hash value), $h = H(m) \in \{0,1\}^n$.
n	digest length in bits (e.g. 256 for SHA-256).
f	compression function, $\{0,1\}^n \times \{0,1\}^r \rightarrow \{0,1\}^n$: (state, block) $\mapsto$ new state.
r	message block size in bits (e.g. 512 for SHA-256).
IV	initialization vector: the fixed n -bit starting state, $S_0 = IV$.

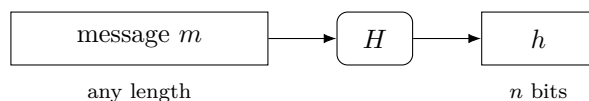
1 What a cryptographic hash is

A *hash function* takes an input of any length and returns a short, fixed-length output:

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

Symbols:

- H — the hash function.
- m — the input, called the *message*: a bit string of arbitrary length, $m \in \{0, 1\}^*$ (the “*” means any number of bits, including very long inputs like a whole file).
- h — the output, called the *digest* (or hash value): $h = H(m)$, exactly n bits, $h \in \{0, 1\}^n$.
- n — the digest length in bits; e.g. $n = 256$ for SHA-256.



Every hash function — cryptographic or not — has three basic traits:

- *Deterministic*: the same m always gives the same h .
- *Fixed-size output*: h is n bits no matter how big m is.
- *Fast*: $H(m)$ is cheap to compute, even for a large m .

1.1 What makes it *cryptographic*

A cryptographic hash adds security guarantees. Anyone can run H forwards, but certain “backwards” or “matching” tasks must be *computationally infeasible* — meaning no known method does much better than brute force. There are three such properties, and keeping them straight is the whole game.

Preimage resistance (one-wayness). A *preimage* of a digest h is any input that hashes to it — “preimage” is the function-theory name for an input that maps to a given output. The property: given h , it is infeasible to find any message m with $H(m) = h$. Going $m \rightarrow h$ is easy; going $h \rightarrow m$ is not (hence *one-way*).

Second-preimage resistance. A *second preimage* is a *second* input that hashes to the same value as a given first one. The property: given a *specific* message m_1 , it is infeasible to find a different $m_2 \neq m_1$ with $H(m_2) = H(m_1)$. The first message is fixed and handed to you; you must collide with *it*.

Collision resistance. A *collision* is any two different messages that hash to the same digest. The property: it is infeasible to find any pair $m_1 \neq m_2$ with $H(m_1) = H(m_2)$. Here you choose *both* messages — which is why this is the easiest of the three to break.

The last two differ only in who picks the messages:

Property	You are given	Find	Generic cost
Preimage	a digest h	any m with $H(m) = h$	2^n
Second-preimage	a message m_1	$m_2 \neq m_1$ with $H(m_2) = H(m_1)$	2^n
Collision	(nothing)	any $m_1 \neq m_2$ with $H(m_1) = H(m_2)$	$2^{n/2}$

Why the costs differ. For an ideal n -bit hash, brute force needs about 2^n tries to hit a *specific* target (preimage, second-preimage), but only about $2^{n/2}$ to find *some* colliding pair — the *birthday bound*. So a 256-bit hash gives $\approx 2^{256}$ preimage security but only $\approx 2^{128}$ collision security; that gap is exactly why digests are made twice as long as the collision security you want. We derive the birthday bound in the attacks section.

The mental model. Treat $h = H(m)$ as a *fingerprint* of the data: short, fixed-size, and practically unique. Preimage resistance says you cannot recover the data from its fingerprint; collision resistance says you cannot find two different inputs that share one — which is what would let someone swap a genuine file for a forgery without changing its hash.

1.2 Symbols

New persistent symbols this section — H, m, h, n — are in the global [list of symbols](#). The symbols below are *local to this section*.

Local symbol	Meaning (this section)
m_1, m_2	two candidate messages in the resistance definitions

2 The Merkle–Damgård construction

§1 treated H as a black box. Now we build one. The trick: start from a function that hashes a *fixed*-size input, and chain it to cover messages of any length.

2.1 The compression function

$$f : \{0, 1\}^n \times \{0, 1\}^r \rightarrow \{0, 1\}^n, \quad (S, M) \mapsto S'.$$

f takes an n -bit *state* S and an r -bit *message block* M , and returns a new n -bit state. New symbols: r — the block size in bits (e.g. $r = 512$ for SHA-256, whose state is $n = 256$); and IV — the *initialization vector*, a fixed n -bit constant baked into the hash.

2.2 Padding

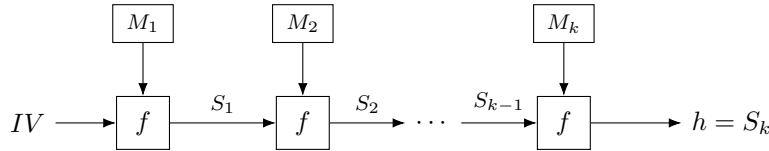
A message is rarely an exact multiple of r bits, so first pad it. The standard scheme (*Merkle–Damgård strengthening*): append a single 1 bit, then enough 0 bits, and finally encode the *original message length* in the last block. That length encoding matters — we lean on it in the security argument below. After padding, the message splits into exactly k blocks $M_1, M_2, \dots, M_k$, each r bits.

2.3 Chaining

Thread an n -bit state through the blocks, starting from IV :

$$S_0 = IV, \quad S_i = f(S_{i-1}, M_i) \quad (i = 1, \dots, k), \quad h = S_k.$$

S_i is the *chaining value* after block i , and the final state S_k is the digest. **Notation flag.** We write the state as S_i (not H_i) on purpose: H is the hash *function* and h the digest, so a subscripted H would collide with both.



2.4 Why it preserves collision resistance

The payoff: *if f is collision-resistant, so is H .* Sketch. Suppose someone finds $m \neq m'$ with $H(m) = H(m')$. Pad both and run the chains, giving states $S_0, \dots, S_k$ and $S'_0, \dots, S'_{k'}$ with $S_0 = S'_0 = IV$ and $S_k = S'_{k'} = h$. Walk *backwards* from the end:

- The last step gives $f(S_{k-1}, M_k) = f(S'_{k'-1}, M'_{k'}) = h$. If the two inputs (S_{k-1}, M_k) and $(S'_{k'-1}, M'_{k'})$ differ, that *is* a collision in f — done.
- If they are equal, then $M_k = M'_{k'}$ and $S_{k-1} = S'_{k'-1}$. Because the last block encodes the length, equal last blocks force $|m| = |m'|$, hence $k = k'$. Recurse on the previous block.

Since $m \neq m'$, the recursion must reach a block where the messages first differ while their output states are equal — exactly a collision in f . So any collision in H yields one in f : if none can be found for f , none can be found for H .

2.5 A built-in weakness: length extension

Because the digest *is* the final state S_k , anyone who knows $h = H(m)$ and the length $|m|$ can set S_k as their starting state and keep chaining more blocks — computing $H(m \parallel \text{pad} \parallel x)$ for a chosen x *without knowing* m . This breaks the naive “secret-prefix” MAC $H(\text{key} \parallel m)$. We give this attack its own treatment — with a real-world break and the fix — in §5.

2.6 Symbols

New persistent symbols this section — f, r, IV — are in the global [list of symbols](#). The symbols below are *local to this section*.

Local symbol	Meaning (this section)
M_i	the i -th r -bit message block (after padding)
S_i	chaining value (state) after block i ; $S_0 = IV$, $S_k = h$
k	number of blocks after padding
i	block index, $1, \dots, k$

3 Popular hash algorithms

A quick map of the named hash functions you actually meet. They fall into three construction families: *Merkle–Damgård* (§2), the *sponge*, and *tree* hashing. Later sections open up the internals; this is the overview.

3.1 The Merkle–Damgård line

These chain a compression function exactly as in §2, so all three inherit its length-extension quirk.

- **MD5** (1992) — 128-bit digest. Collisions are now trivial to produce (§4); usable only as a non-security checksum, never for integrity or signatures.
- **SHA-1** (1995) — 160-bit. Collision-broken in practice (the 2017 “SHattered” attack); deprecated everywhere.
- **SHA-2** (2001) — the family SHA-224/256/384/512. **SHA-256** ($n = 256$) and **SHA-512** ($n = 512$) are today’s workhorses, securing TLS, Bitcoin, Git’s newer mode, and much more. No practical weakness.

3.2 Beyond Merkle–Damgård

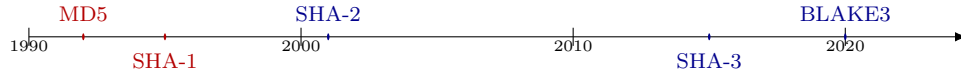
Two modern designs drop the chain — and with it, length extension.

- **SHA-3** / Keccak (2015) — built on a *sponge*: the message is “absorbed” into a large internal state, then output bits are “squeezed” out. Because the full state is never exposed, length extension is impossible. It also offers *XOF*s (SHAKE128/256) that emit *any* requested number of bits.
- **BLAKE2** / **BLAKE3** — very fast hashes from the ChaCha lineage. **BLAKE3** (2020) hashes the input as the leaves of a *Merkle tree* (§9), so it parallelizes across cores; it is an *XOF*, 256-bit by default, and typically faster than SHA-256.

The jargon: a *sponge* absorbs the message into a fixed large state and then squeezes out digest bits; an *XOF* (extendable-output function) is a hash whose output length *you* choose; *tree hashing* splits the input into chunks hashed as Merkle leaves, for parallelism.

The ChaCha lineage. ChaCha (and its predecessor Salsa20), by Daniel J. Bernstein, is a fast *stream cipher* — encryption, not hashing — but BLAKE2/BLAKE3 borrow its core mixing, which is why they are grouped with it. ChaCha is an *ARX* design: built only from **Add** (modulo 2^{32}), bitwise **Rotate**, and **XOR** — three cheap word operations that run in *constant time* with no table lookups, so they leak nothing through cache-timing side channels (a weakness that has bitten table-based ciphers like AES). The basic step, a “quarter-round”, stirs four 32-bit words of a 512-bit state; ChaCha20 applies it for 20 rounds to turn a key and nonce into keystream. BLAKE’s compression function is essentially this same ARX permutation — which is where BLAKE3’s speed comes from. (ChaCha20 as a cipher is an encryption topic in its own right; here we care only that BLAKE inherits its mixing.)

Algorithm	Year	Digest n	Construction	Status
MD5	1992	128	Merkle–Damgård	broken; checksums only
SHA-1	1995	160	Merkle–Damgård	broken; deprecated
SHA-2	2001	256/512	Merkle–Damgård	secure (workhorse)
SHA-3	2015	256/512, XOF	sponge	secure; no length-ext.
BLAKE3	2020	256, XOF	tree	secure; fast, parallel



Red = collision-broken; blue = secure.

Takeaways. Construction drives the security story: the Merkle–Damgård trio (MD5, SHA-1, SHA-2) all length-extend, and the two oldest are collision-broken; the sponge (SHA-3) and tree (BLAKE3) designs avoid length extension by never exposing the full state. For new work reach for SHA-256, SHA-3, or BLAKE3 — never MD5 or SHA-1.

3.3 Symbols

No new symbols — this section is a descriptive survey, using n (digest length, §1) throughout.

4 Attacks and resistance

§1 quoted two costs — 2^n to break preimage resistance and $2^{n/2}$ for collisions — and §2 promised the derivation. Here it is. Throughout we model an ideal hash as a *random function*: for a fresh input, H returns a value drawn uniformly and independently from the 2^n possible digests. Real hashes only approximate this, but it sets the bar an attacker must beat.

Quick recall of the three targets from §1, in plain words:

- **preimage** — given a digest, find *any* input that produces it (i.e. run the hash backwards / invert it);
- **second preimage** — given one message, find a *different* message with the same digest;
- **collision** — find *any* two different messages that share a digest.

The first two chase a *fixed* target; the third lets the attacker pick *both* messages, which is why it comes cheaper. Now the costs.

4.1 Brute-force preimage: $\approx 2^n$

Fix a target digest h . The attacker tries guesses $m_1, m_2, \dots$ and checks whether $H(m_j) = h$. Under the model each guess hashes to a uniform digest, so

$$p := \Pr[H(m_j) = h] = \frac{1}{2^n} = 2^{-n}.$$

The trials are independent, so the number of guesses until the first hit is geometric, with expected value

$$\mathbb{E}[\text{guesses}] = \frac{1}{p} = 2^n.$$

Nothing beats this for an ideal hash, because the target is *fixed* — every guess is an independent 2^{-n} shot. Second-preimage is the same: the target $H(m_1)$ is just one specific value. For $n = 256$ that is 2^{256} hashes — beyond any computation that will ever run.

4.2 Collisions and the birthday bound: $\approx 2^{n/2}$

Now the attacker chases no fixed target — *any* two coinciding digests will do. Hash q distinct messages, getting digests $h_1, \dots, h_q$. Two steps.

One pair. For a single pair $j \neq l$, both digests are uniform and independent, so

$$\Pr[h_j = h_l] = 2^{-n}.$$

All pairs. There are

$$\binom{q}{2} = \frac{q(q-1)}{2} \approx \frac{q^2}{2}$$

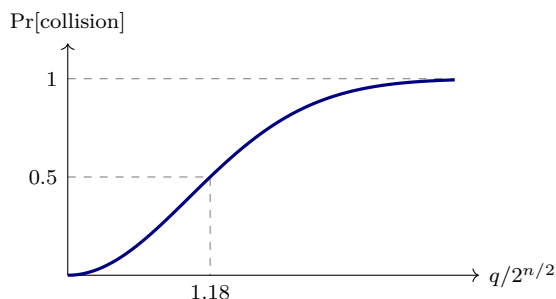
pairs. By linearity of expectation (add the per-pair probability over all pairs), the expected number of colliding pairs is

$$\mathbb{E}[\text{collisions}] \approx \frac{q^2}{2} \cdot 2^{-n} = \frac{q^2}{2^{n+1}}.$$

A collision becomes likely once this reaches about 1:

$$\frac{q^2}{2^{n+1}} \approx 1 \implies q^2 \approx 2^{n+1} \implies q \approx 2^{(n+1)/2} \approx 2^{n/2}.$$

A more careful count gives $\Pr[\text{some collision}] \approx 1 - e^{-q^2/2^{n+1}}$, which passes 50% at $q \approx 1.18 \cdot 2^{n/2}$:



Why the square root. q hashes create $\sim q^2$ pairs, so you need only $q \sim \sqrt{2^n} = 2^{n/2}$ messages to amass $\sim 2^n$ pairs — enough that one collides. Preimage search has a single fixed target, so q guesses give only q chances, not q^2 . (It is the birthday paradox: 23 people share a birthday with probability $> \frac{1}{2}$ because $\binom{23}{2} = 253$ pairs run against only 365 days.)

4.3 What this means for digest size

Collision resistance is only $\approx n/2$ bits; preimage resistance is $\approx n$ bits. So size the digest at *twice* the collision security you want — 128-bit collision security needs $n = 256$, i.e. SHA-256.

Digest n	Preimage $\approx 2^n$	Collision $\approx 2^{n/2}$	Status
128 (MD5)	2^{128}	2^{64}	collisions broken
160 (SHA-1)	2^{160}	2^{80}	collisions broken
256 (SHA-256)	2^{256}	2^{128}	secure
512 (SHA-512)	2^{512}	2^{256}	secure

4.4 Real-world breaks

These are not hypothetical, and the pattern is consistent: *collision* resistance falls long before *preimage* resistance.

- **MD5** ($n = 128$, so 2^{64} birthday): collisions found in 2004, now computable in seconds. *Chosen-prefix* collisions forged a Microsoft code-signing certificate (the Flame malware, 2012).
- **SHA-1** ($n = 160$, 2^{80} birthday): the 2017 “SHAttered” attack produced two different PDFs with the same SHA-1 digest at about 2^{63} work. Now deprecated everywhere.
- Both remain *preimage*-resistant in practice — you still cannot invert a given MD5 digest — which is exactly why collision resistance is the first thing to go.

Use SHA-256 (or SHA-3); avoid MD5 and SHA-1 for anything security-relevant.

4.5 Symbols

No new persistent symbols this section — it is analysis built on H, h, n . Local symbols:

Local symbol	Meaning (this section)
q	number of messages hashed (queries) during an attack
p	per-guess probability of matching a fixed target, $p = 2^{-n}$
$\mathbb{E}[\cdot]$	expected count (via linearity of expectation)
j, l	indices over the q hashed messages

5 Length-extension attacks

The Merkle–Damgård chain (§2) carries one notorious weakness, flagged when we built it: *length extension*. It earns its own section, because it is subtle, it broke real systems, and it is the reason HMAC (§6) exists.

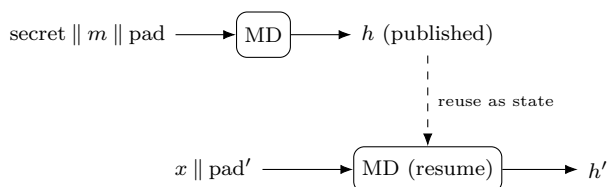
5.1 What it is

In a Merkle–Damgård hash the digest *is* the final chaining state S_k (§2). So if you know $h = H(m)$ and the length $|m|$, you can set h as the state and *keep hashing*: with no knowledge of m itself, you compute

$$H(m \parallel \text{pad} \parallel x)$$

for any suffix x you choose, where pad is the (public, deterministic) padding that H applied to m . You obtain a valid digest of a *longer* message you never fully knew.

Why only one pad in the formula? Two paddings are really in play. The inner pad is the original padding of m — here it counts as genuine *message* bytes, included precisely so the chaining state after $m \parallel \text{pad}$ equals h . Then the outer H finalises as it always does, appending *its own* padding over the *new* total length $|m \parallel \text{pad} \parallel x|$ — that is the pad' in the diagram below. So x *is* padded; that padding lives inside the $H(\cdot)$ notation. Writing pad' explicitly in the argument would be wrong — it would pad twice.



The published digest h is reused as the MD state; the attacker hashes chosen x from there to get a valid digest h' of $\text{secret} \parallel m \parallel \text{pad} \parallel x$ — with no secret.

You need the public padding rule (everyone has it) and the length $|m|$; the content of m stays unknown, and even $|m|$ is short and brute-forceable.

5.2 Why it is harmful

A bare hash has no secret, so “extending a digest” sounds harmless. The damage appears when a hash is misused for *authentication* as a *secret-prefix MAC*:

$$\text{tag} = H(\text{secret} \parallel \text{message}).$$

This looks fine — only a secret-holder can compute the tag. But length extension breaks it: from one valid (message, tag) pair, an attacker forges a valid tag for $\text{message} \parallel \text{pad} \parallel x$ — appending attacker-chosen data — *without ever learning the secret*. The secret sits in the untouched prefix, so the extended input is still “secret || (something)”, a legitimate tag for that something.

5.3 A real attack: forged API signatures

This is not hypothetical. Around 2009 several web APIs (Flickr’s among the best known) authenticated requests with exactly $\text{MD5}(\text{secret} \parallel \text{params})$. The attack (Duong and Rizzo):

1. The attacker holds one legitimate request: its parameters and signature $\text{sig} = \text{MD5}(\text{secret} \parallel \text{params})$.
2. They choose malicious extra parameters x — say ones that escalate permissions or change the method called.
3. By length extension, starting from sig they compute $\text{sig}' = \text{MD5}(\text{secret} \parallel \text{params} \parallel \text{pad} \parallel x)$ — guessing the short secret’s *length*, never its value.
4. They submit the request with parameters $\text{params} \parallel \text{pad} \parallel x$ and signature sig' . The server recomputes $\text{MD5}(\text{secret} \parallel \text{those params})$, gets sig' , and **accepts the forged request**.

The “glue” pad shows up as junk bytes among the parameters, which lax parsers ignored. Off-the-shelf tools (e.g. `hashpump`) automate the whole thing, and the same bug recurred across many sites that rolled their own secret-prefix MAC.

5.4 The fix

- Use **HMAC** (§6) instead of a raw secret-prefix hash — its nested, double-keyed structure exposes no extendable final state.
- Or use a hash that is not length-extendable: **SHA-3**’s sponge (§11) or **BLAKE3** (§12), which never reveal the full internal state; or a truncated hash such as SHA-512/256.

Never authenticate with $H(\text{secret} \parallel m)$.

5.5 Symbols

No new persistent symbols. Local:

Local symbol	Meaning (this section)
<code>secret</code>	the shared secret key of a (broken) secret-prefix MAC
x	attacker-chosen data appended by the extension
<code>pad</code>	the deterministic Merkle–Damgård “glue” padding
<code>sig</code>	the MAC tag / signature

6 HMAC: keyed authentication

So far H has no secret — anyone can compute it. A *message authentication code* (MAC) adds a shared secret key k so that a tag proves two things at once: the message was not tampered with (integrity) *and* it came from someone holding the key (authenticity).

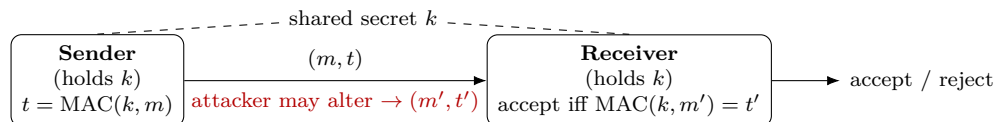
6.1 What a MAC is

Formally a MAC is a pair of algorithms sharing the secret key k — one to make a tag, one to check it:

$$\begin{aligned} \text{tag: } t &= \text{MAC}(k, m) \in \{0, 1\}^\ell, \\ \text{verify: } \text{Vrf}(k, m, t) &= \begin{cases} \text{accept} & \text{if } t = \text{MAC}(k, m), \\ \text{reject} & \text{otherwise.} \end{cases} \end{aligned}$$

The *tag* t is a fixed-length, *keyed* fingerprint — like the digest $h = H(m)$ of §1, but ℓ bits ($\ell = n$ for HMAC) and *unforgeable without k* . Verification simply recomputes the tag and compares.

In use, the two parties share k in advance. The sender appends $t = \text{MAC}(k, m)$ and transmits (m, t) ; on the wire an attacker may read or alter it to (m', t') ; the receiver, holding k , accepts only if $\text{MAC}(k, m') = t'$:



Any tampering is caught, because forging the matching tag for a changed message needs k . **Security goal (unforgeability):** even after seeing many valid (m, t) pairs, an attacker cannot produce a valid tag for *any* new message. The naive constructions below fail exactly this goal.

6.2 Why the obvious constructions fail

Length extension (§2) has exactly one move: append data to the *end* of whatever was hashed and still get a valid hash — the tag *is* the running state, so you just keep running from it. What decides whether that breaks a MAC is *where the secret sits relative to the end*.

Secret-prefix, $t = H(k \parallel m)$ — broken. The secret is at the front, so extension never touches it. From t the attacker computes

$$t' = H(k \parallel m \parallel \text{pad} \parallel x),$$

and regrouping reveals what it is: $k \parallel \underbrace{(m \parallel \text{pad} \parallel x)}_{m'}$, i.e. $H(k \parallel m') = \text{MAC}(k, m')$ — a *valid* tag for

the new message m' . The key never moved, so the input is still a legitimate “ $k \parallel \text{message}$ ”. The attacker needs the *length* $|k|$ (the glue “pad” encodes $|k| + |m|$, and only $|m|$ is known) but never its *value*; key lengths are short and standard, or just brute-forced over a few dozen tries. Forgery.

Secret-suffix, $t = H(m \parallel k)$ — **safe from extension, broken another way**. Now the secret is at the end. Extension still gives

$$t' = H(m \parallel k \parallel \text{pad} \parallel x),$$

but a valid suffix-tag must *end* in k , and this string ends in the attacker's x with k stranded in the middle — so it is the tag of no message at all. Extension is useless here. The catch is a *different* weakness: secret-suffix leans on collision resistance, the first property to fall (§4). An *offline* collision $m \neq m'$ with $H(m) = H(m')$ (found with no key) gives $H(m \parallel k) = H(m' \parallel k)$, the same tag for both — a forgery once m is signed.

HMAC sidesteps both.

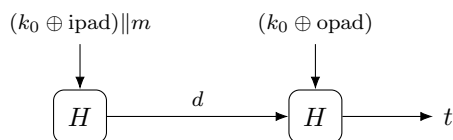
6.3 The construction

First normalize the key to one block (r bits, §2): if k is longer than r bits, replace it with $H(k)$; then zero-pad to exactly r bits. Call the result k_0 . Fix two r -bit pad constants — *ipad* (the byte 0x36 repeated) and *opad* (the byte 0x5c repeated). Then

$$\text{HMAC}(k, m) = H\left((k_0 \oplus \text{opad}) \parallel H((k_0 \oplus \text{ipad}) \parallel m)\right),$$

two nested hashes. Naming the inner result d :

$$d = H((k_0 \oplus \text{ipad}) \parallel m) \in \{0, 1\}^n, \quad t = H((k_0 \oplus \text{opad}) \parallel d) \in \{0, 1\}^n.$$



Inner hash, keyed by ipad, digests the message to the n -bit d ; the outer hash, keyed by opad, wraps d into the tag t .

6.4 Why it works

Length extension is defeated. The tag is itself a hash, $t = H((k_0 \oplus \text{opad}) \parallel d)$, so an attacker *can* length-extend it to $H((k_0 \oplus \text{opad}) \parallel d \parallel \text{pad} \parallel x)$. But that value is not the HMAC of *any* message: a real HMAC always has outer input $(k_0 \oplus \text{opad}) \parallel$ (an n -bit inner digest), whereas the extended string is longer and not of that form. The receiver recomputes from k and m' and never lands on it, so the forgery is rejected.

Two independent keys. Since $\text{ipad} \neq \text{opad}$, the two derived keys $k_0 \oplus \text{ipad}$ and $k_0 \oplus \text{opad}$ differ, so the inner and outer hashes behave as if keyed independently — they cannot be made to cancel.

It does not rely on collision resistance. HMAC is a provably secure MAC as long as the *compression function* f (§2) behaves like a pseudorandom function — a weaker assumption than collision resistance. This is why HMAC-SHA-1, and even HMAC-MD5, remain practically safe for authentication despite those hashes' broken *collision* resistance (§4): forging an HMAC tag is a different, harder problem than finding a hash collision.

6.5 In practice

HMAC-SHA-256 is everywhere: TLS, JWTs, API request signing, PBKDF2 (next section). Verify tags with a *constant-time* comparison — a byte-by-byte early-exit check leaks, through timing, how many leading bytes matched, which can be enough to forge a tag.

6.6 Symbols

No new persistent symbols — HMAC is built on H, m, h, n, r . Local to this section:

Local symbol	Meaning (this section)
k	the shared secret key (any length)
k_0	key normalized to r bits ($H(k)$ if too long, then zero-padded)
ipad, opad	inner/outer pads: bytes <code>0x36</code> , <code>0x5c</code> repeated to r bits
$\oplus$	bitwise XOR
$\parallel$	concatenation
d	inner digest, $d = H((k_0 \oplus \text{ipad}) \parallel m)$
t	the MAC tag, $\in \{0, 1\}^\ell$ (for HMAC, $\in \{0, 1\}^n$)
ℓ	tag length in bits ($\ell = n$ for HMAC)
Vrf	verification: accept iff the recomputed tag matches

7 Key derivation functions (KDFs)

A key is rarely handed to you in usable form. A password is low-entropy and variable-length; a Diffie–Hellman shared secret is high-entropy but not uniform; and often you need *several* independent keys from one master secret. A *key derivation function* turns input keying material into one or more cryptographic keys of exactly the length you need.

7.1 The general shape

$$K = \text{KDF}(\text{IKM}, \text{salt}, \text{info}, L) \in \{0,1\}^L,$$

with

- IKM — input keying material (the password, shared secret, master key);
- salt — a non-secret randomizer (also used in §8);
- info — an optional context label, so distinct labels yield *independent* keys from the same IKM;
- L — the desired output length in bits.



A KDF is deterministic and *one-way* (the key reveals nothing about IKM). Which KDF you reach for depends on *what the IKM is*.

7.2 High-entropy material: HKDF (extract-then-expand)

When IKM already has plenty of entropy but is not uniform — e.g. a Diffie–Hellman secret, whose bits are not evenly random — the danger is non-uniformity, not guessing. HKDF (built on HMAC, §6) fixes it in two stages.

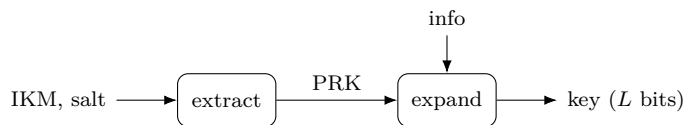
Extract — condense the messy entropy into one uniform pseudorandom key:

$$\text{PRK} = \text{HMAC}(\text{salt}, \text{IKM}) \in \{0,1\}^n.$$

Expand — stretch PRK to L bits, chaining HMAC and folding in info (so the output is bound to its purpose):

$$T_0 = (\text{empty}), \quad T_i = \text{HMAC}(\text{PRK}, T_{i-1} \parallel \text{info} \parallel i),$$

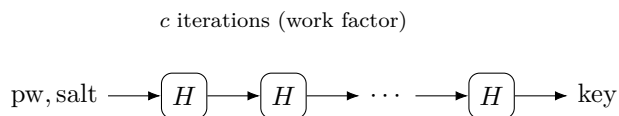
and the derived key is $T_1 \parallel T_2 \parallel \dots$ truncated to L bits.



7.3 Low-entropy material: password-based KDFs (deliberately slow)

When the IKM is a *password*, the danger is the opposite: it is guessable, and the hash is fast (§1), so an attacker can try billions of candidates per second. A password-based KDF defends by being *deliberately expensive*, tuned by a *work factor* c , and salted.

PBKDF2 — iterate. Apply HMAC (§6) c times in a chain:



Cost scales linearly with c (e.g. $c = 600,000$). Choose c so one derivation takes ~ 0.1 – 1 second: trivial once at login, but it cuts an attacker’s billion guesses per second down to a few thousand.

Memory-hardness — beat GPUs and ASICs. PBKDF2 only burns CPU, which is cheap to parallelize on GPUs/ASICs. **scrypt** and **Argon2** (recommended; winner of the Password Hashing Competition) instead force each guess to occupy a large block of memory, expensive to replicate across thousands of cores; they take separate *time*, *memory*, and *parallelism* parameters. **bcrypt** is an older adaptive option.

7.4 Symbols

No new persistent symbols — KDFs are built on H and HMAC. Local to this section:

Local symbol	Meaning (this section)
IKM	input keying material (password, shared secret, master key)
salt	non-secret randomizer
info	context label; distinct labels give independent keys
L	desired output length in bits
PRK	pseudorandom key from HKDF-Extract, n bits
T_i	i -th block of HKDF-Expand output
c	work factor (iteration count) of a password KDF

8 Storing passwords

A server must check passwords without storing them in the clear — a breach should not hand over everyone’s password. The instinct from §1 is to store the digest $H(\text{password})$ and, at login, compare $H(\text{entered})$ against it. Preimage resistance means the stored digest does not reveal the password. But raw $H(\text{password})$ is badly broken in practice, for three reasons; the fixes are *salting* and *slow KDFs*.

8.1 Why raw $H(\text{password})$ fails

Hashes are fast — so guessing is fast. A cryptographic hash is deliberately quick (§1), and passwords carry little entropy (people reuse common ones). An attacker who steals the stored digests runs H over a dictionary, or over all short strings, at billions of guesses per second on a GPU, and recovers most passwords. Preimage resistance is irrelevant here: they are not inverting H , merely trying likely inputs.

Determinism enables precomputation. H is fixed and unkeyed, so an attacker can hash a huge guess list *once*, store the (guess, digest) pairs, and then crack *any* stolen digest by table lookup — instantly, and across every site at once. (Rainbow tables are a space-saving version of this.)

Identical passwords give identical digests. Two users with the same password store the same digest, so the attacker spots the duplicates and one crack breaks all of them.

8.2 Salting: kill precomputation and duplicates

Pick a *salt* s : a random, per-user value (say 128 bits), and store

$$(s, H(s \parallel \text{password}))$$

instead of $H(\text{password})$. The salt is *not* secret — it sits in the clear next to the digest. What it buys:

- **No precomputation.** A precomputed table would have to be rebuilt *per salt*; with 128-bit random salts there are far too many, so the attacker must hash guesses freshly against each target.
- **No duplicates.** Same password + different salts $\rightarrow$ different digests, so duplicates vanish and one crack no longer breaks others.

What salting does *not* do: slow down a single guess. Against one target with a fast hash, the attacker still grinds billions of guesses per second — which is the next fix.

Pepper — a secret kept out of the database. A *pepper* is like a salt with two differences: it is *secret*, and it is *not* stored in the database — it lives in the application config, an environment variable, or a hardware security module, separate from the digests. It is usually a single site-wide value, not per-user. Apply it as a key on a keyed hash, e.g. store $\text{HMAC}_{\text{pepper}}(s \parallel \text{password})$ (§6). The point: if *only* the database leaks (say through SQL injection), the attacker has the salts and digests but *not* the pepper, so cannot even begin guessing. The catch — and why it is defence-in-depth, not a replacement for salt + slow KDF: it helps only if the pepper is *not* stolen too. A full server compromise that grabs the app config gets the pepper as well, and then it adds nothing.

Rotating a pepper is also awkward, since you cannot recompute the stored values without the plaintext passwords.

8.3 Slow it down with a password KDF

Salting still leaves a *fast* hash, so an attacker can grind billions of guesses per second against a single target. The remedy is to derive the stored value with a slow, salted *password-based KDF* (§7) — PBKDF2, or a memory-hard **scrypt**/**Argon2** — whose tunable work factor makes each guess expensive. Store

$$(\text{salt}, \text{KDF parameters}, \text{KDF}(\text{password}, \text{salt}, \text{params})),$$

and verify a login by recomputing and comparing in constant time (§6). Each defence answers one specific attack:

Attack	Defense
Fast brute force / dictionary	slow password KDF (§7)
Precomputation / rainbow tables	per-user random salt s
Identical passwords $\rightarrow$ identical digests	per-user random salt s
GPU / ASIC parallelism	memory-hard KDF (Argon2, scrypt)
Database-only breach (app secret intact)	secret pepper, kept out of the DB

8.4 Symbols

No new persistent symbols. Local to this section:

Local symbol	Meaning (this section)
pw	the user's password (the low-entropy secret to protect)
s	salt: a random, per-user value, stored in the clear
pepper	a secret, site-wide value kept <i>out</i> of the database

9 Merkle trees

Suppose you have many items — file chunks, log entries, a block’s transactions — and want a *single* short value committing to all of them, plus a cheap way to prove any one item is included without handing over the rest. A *Merkle tree* (hash tree) does both, using only H .

9.1 Construction

Start with N data items $d_0, \dots, d_{N-1}$ (take N a power of 2 for now). Hash each into a *leaf*:

$$L_i = H(d_i), \quad i = 0, \dots, N-1.$$

Then build upward: every parent is the hash of its two children concatenated,

$$\text{parent} = H(\text{left} \parallel \text{right}),$$

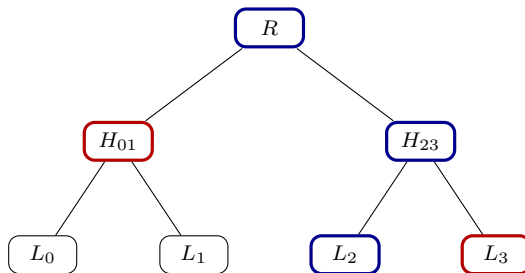
halving the node count at each level until one remains — the *root* R , a single n -bit commitment to all N items. (If N is not a power of 2, pad by duplicating the last node on each short level.)

Notation flag. N (uppercase) is the number of leaves; n (lowercase) is the digest length in bits from §1 — different quantities.

Change any single item and its leaf changes, which changes its parent, and so on up to R . By collision resistance (§1) you cannot alter an item without altering the root, so R pins down every item at once.

9.2 Proving membership

The payoff: to prove d_i sits in the tree with root R , you do *not* send all N items. You send only the *sibling* hash at each level along the path from leaf i up to the root — the *authentication path*, just $\log_2 N$ hashes. The verifier recomputes $L_i = H(d_i)$, combines it with each supplied sibling (in the correct left/right order) up the tree to a candidate root R' , and accepts iff $R' = R$.



Merkle tree over four items ($L_i = H(d_i)$; each internal node is $H(\text{left} \parallel \text{right})$). To prove d_2 under root R , supply the two red siblings L_3 and H_{01} (the authentication path). The verifier recomputes the blue nodes — $L_2 = H(d_2)$, then $H_{23} = H(L_2 \parallel L_3)$, then $R' = H(H_{01} \parallel H_{23})$ — and accepts iff $R' = R$. Two hashes for four leaves; $\log_2 N$ in general, so a tree of a million items needs only 20.

9.3 A tiny example

Take four items with a *toy* 1-byte hash — two hex digits, where a real digest would be 32 bytes, so the values here are purely illustrative. Hash the leaves, then the parents, then the root:

$$L_0 = 3\text{f}, \quad L_1 = \text{a}1, \quad L_2 = 7\text{c}, \quad L_3 = 0\text{e};$$

$$H_{01} = H(3f \parallel a1) = b8, \quad H_{23} = H(7c \parallel 0e) = 5d, \quad R = H(b8 \parallel 5d) = 2a.$$

Now prove d_2 belongs, given only the root $R = 2a$. The prover sends d_2 and its two authentication-path siblings: the right sibling $L_3 = 0e$ and the left sibling $H_{01} = b8$. The verifier walks up:

1. $L_2 = H(d_2) = 7c$;
2. $H'_{23} = H(L_2 \parallel L_3) = H(7c \parallel 0e) = 5d$;
3. $R' = H(H_{01} \parallel H'_{23}) = H(b8 \parallel 5d) = 2a$;
4. $R' = 2a = R$, so **accept** — d_2 is in the tree.

Two sibling hashes settled it ($\log_2 4 = 2$); d_0 and d_1 were never touched. And the left/right order matters: L_2 sits *left* of L_3 , and H_{01} *left* of H'_{23} — feed them in the wrong order and R' would not match.

9.4 The same proof at real scale

That toy proof is exactly what a cryptocurrency *light wallet* does — only with thousands of leaves instead of four. A block bundles, say, 3,000 transactions, but its small header stores only their Merkle *root*. Your phone wallet wants to confirm “my payment tx is in block #850000” without downloading the whole block (~ 1 MB), let alone the whole chain (hundreds of GB).

So it asks a full node for a *Merkle proof* of tx : the node returns tx plus the authentication path — about $\log_2 3000 \approx 12$ sibling hashes, i.e. $12 \times 32 = 384$ bytes. The wallet hashes tx up those 12 levels to a candidate root and checks it against the root in the block header (which it already trusts, since the header is sealed by the chain’s proof-of-work). A match proves tx is in the block — settled with ~ 400 bytes and 12 hashes instead of a megabyte. That is the toy d_2 proof, scaled to 3,000 leaves and real SHA-256.

9.5 Where they are used

Beyond that wallet, the same membership-proof idea appears in:

- **Git**: tree objects are hashes of their contents, so one commit hash commits to the entire file tree (content addressing).
- **Certificate Transparency**: append-only logs of issued certificates use Merkle inclusion and consistency proofs.
- **Storage and sync** (ZFS, Btrfs, IPFS, BitTorrent): verify chunks against a root hash and pinpoint exactly which chunk is corrupt.

9.6 Symbols

No new persistent symbols — Merkle trees are an application of H . Local to this section:

Local symbol	Meaning (this section)
d_i	the i -th data item (leaf input)
N	number of leaves / items (a power of 2); <i>not</i> the digest bits n
L_i	leaf hash, $L_i = H(d_i)$
internal node	$H(\text{left} \parallel \text{right})$ of its two children
R	the root: an n -bit commitment to all N items

10 Inside SHA-2 (SHA-256)

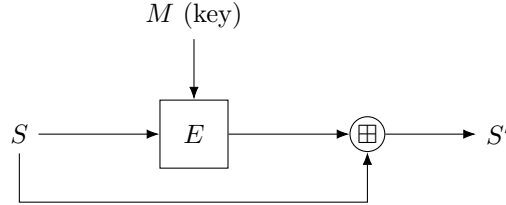
SHA-256 is the Merkle–Damgård hash of §2 with a specific compression function f . Recall the chain from §2: f takes the running *state* S (the $n = 256$ -bit chaining value, 8 words of 32 bits) and one *message block* M (the $r = 512$ -bit chunk being absorbed, 16 words), and returns the next state, $f(S, M) = S'$. This section opens up f . Throughout, $\boxplus$ is addition modulo 2^{32} (words wrap around), and every variable is a 32-bit word.

10.1 The compression function: a block cipher plus feed-forward

f has the *Davies–Meyer* shape:

$$f(S, M) = E_M(S) \boxplus S,$$

where E_M is a 256-bit keyed permutation (a block cipher, named SHACAL-2) whose *key* is the message block M , and the input state S is added back at the end (the *feed-forward*). That feed-forward is what makes f one-way: even knowing M you cannot run E_M backward to recover S , because the final $\boxplus S$ folds S in non-invertibly. (A pure permutation would be reversible — and a reversible compression function cannot be collision-resistant.)



10.2 The message schedule

First, “word” means a 32-bit chunk. The compression runs 64 rounds (the round function below), and *each round consumes one 32-bit word*, written W_t — so it needs 64 words in all, $W_0, \dots, W_{63}$. But the message block M supplies only sixteen of them: M is 512 bits = 16×32 , i.e. 16 words. The *message schedule* bridges that gap — it *expands* the 16 block words into the 64 the rounds need. The first sixteen *are* the block, $W_0, \dots, W_{15}$ (this is where your message data enters); each later word is mixed from four earlier ones:

$$W_t = \sigma_1(W_{t-2}) \boxplus W_{t-7} \boxplus \sigma_0(W_{t-15}) \boxplus W_{t-16}, \quad 16 \leq t \leq 63,$$

where σ_0, σ_1 stir a word with rotations and a shift:

$$\sigma_0(x) = \text{ROTR}^7 x \oplus \text{ROTR}^{18} x \oplus \text{SHR}^3 x, \quad \sigma_1(x) = \text{ROTR}^{17} x \oplus \text{ROTR}^{19} x \oplus \text{SHR}^{10} x.$$

(ROTR^k rotates the 32 bits right by k ; SHR^k shifts right by k , dropping the low bits.) This spreads every message bit across many rounds.

10.3 The round function

At the start of a block, the eight working words a, b, c, d, e, f, g, h are *loaded from the incoming state*: $(a, b, \dots, h) \leftarrow S$. (For the very first block S is the fixed IV; for every later block it is the chaining value handed over from the previous block — so these starting words are identical only at the very beginning.) Then 64 rounds stir them. Round t forms two quantities,

$$\begin{aligned} T_1 &= h \boxplus \Sigma_1(e) \boxplus \text{Ch}(e, f, g) \boxplus K_t \boxplus W_t, \\ T_2 &= \Sigma_0(a) \boxplus \text{Maj}(a, b, c), \end{aligned}$$

then shifts the registers down and injects T_1, T_2 :

$$h = g, \quad g = f, \quad f = e, \quad e = d \boxplus T_1, \quad d = c, \quad c = b, \quad b = a, \quad a = T_1 \boxplus T_2.$$

The helper functions:

- $\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$ — “choose”: bit by bit, e picks f or g ;
- $\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$ — the bitwise “majority” of the three;
- $\Sigma_0(x) = \text{ROTR}^2 x \oplus \text{ROTR}^{13} x \oplus \text{ROTR}^{22} x$ and $\Sigma_1(x) = \text{ROTR}^6 x \oplus \text{ROTR}^{11} x \oplus \text{ROTR}^{25} x$;
- K_t — a fixed round constant (the fractional bits of the cube root of the t -th prime).

Ch and Maj are *bitwise* — applied independently to each of the 32 bit positions — so a single-bit truth table pins them down. **Ch** (“choose”) is a multiplexer: the e bit selects, taking f ’s bit where $e = 1$ and g ’s bit where $e = 0$. **Maj** (“majority”) outputs the bit value that at least two of its three inputs agree on.

x	y	z	$\text{Ch}(x, y, z)$	$\text{Maj}(x, y, z)$
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	1	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Read $\text{Ch}(x, y, z)$ as “if x then y else z ” (here $x = e$ is the selector); read $\text{Maj}(x, y, z)$ as “1 when two or more inputs are 1”.

After 64 rounds the eight words are added (feed-forward) to the round’s input state, yielding the new chaining value. The starting *IV* (§2) is the fractional bits of the square roots of the first eight primes.

Everything here is *Add*, *Rotate*, *XOR* plus the boolean Ch/Maj — no table lookups, so SHA-256 runs in constant time. Its security rests on 64 rounds of this mixing; no known attack beats the generic 2^{128} collision bound (§4).

10.4 Padding: filling the last block

A message is almost never an exact multiple of 512 bits, so SHA-256 pads it up to one before hashing (this is §2’s Merkle–Damgård strengthening, made concrete). To a message of ℓ bits it appends, in order: a single 1 bit; then k 0 bits; then ℓ written as a 64-bit big-endian integer. The count k is the smallest that makes the total a multiple of 512:

$$\ell + 1 + k + 64 \equiv 0 \pmod{512}.$$

The padded message now divides evenly into 512-bit blocks, and the final block carries the message tail, the marker bit, the zeros, and the length:

Local symbol	Meaning (this section)
$\boxplus$	addition modulo 2^{32}
$a, \dots, h$	the eight 32-bit working words of the state
E_M	256-bit keyed permutation (SHACAL-2), $\text{key} = M$
W_t, K_t	schedule word / round constant for round t
Ch, Maj	“choose” and “majority” bitwise functions
$\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$	rotate/shift mixing functions
$\text{ROTR}^k, \text{SHR}^k$	rotate-right / shift-right by k bits
t	round index, $0, \dots, 63$

11 Inside SHA-3 (the sponge)

SHA-3 throws away the Merkle–Damgård chain. It is a *sponge*: a single fixed permutation stirring a large state, with the message poured in and the digest wrung out.

11.1 State, rate, and capacity

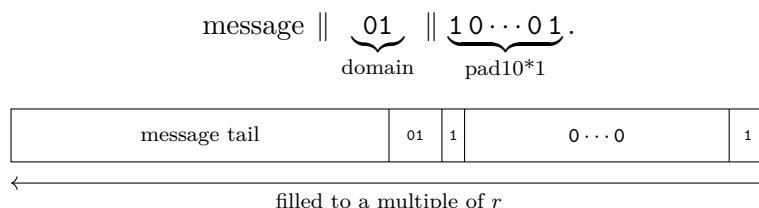
The state is 1600 bits, split into two parts:

$$\underbrace{r}_{\text{rate}} + \underbrace{c}_{\text{capacity}} = 1600.$$

Only the *rate* r bits ever touch the message; the *capacity* c bits are a hidden security reserve, never read out. Security is about $c/2$ bits, so SHA3-256 takes $c = 512$ (hence $r = 1088$), giving the 2^{128} collision resistance expected of a 256-bit hash.

11.2 Padding to the rate

The message must be padded to a whole number of *rate*-sized blocks — note: to r bits (1088 for SHA3-256), not to 512. SHA-3 uses the rule *pad10*1* (read “one, then zero or more zeros, then one”), preceded by two domain-separation bits. To the message it appends, in order: the bits 01 (marking “this is SHA-3”, not its XOF cousin SHAKE, §3); a 1; the fewest 0s; and a final 1 — so the total becomes a multiple of r :



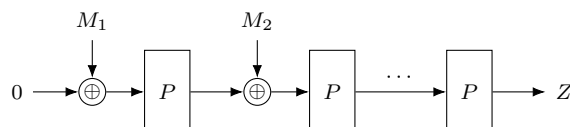
The two framing 1 bits make the padding *injective* (no two messages pad alike) and ensure the final block is never all-zero. As with SHA-2, if the tail leaves too little room for the markers, the padding spills into one extra block. And the domain-separation bits let the *same* permutation P safely serve both SHA-3 (01) and the SHAKE XOFs (1111).

11.3 Absorb, then squeeze

Let P denote the permutation (Keccak- f , below) and S the full 1600-bit state. *Absorb*: take the padded message (above) cut into r -bit blocks $M_1, M_2, \dots$. The state starts all-zero, $S = 0^{1600}$ (there is no IV). For each block, XOR it into the *rate* — the top r bits, the block padded with c zeros so the XOR leaves the capacity untouched — then apply P :

$$S \leftarrow P(S \oplus (M_i \parallel 0^c)).$$

Squeeze: read the first r bits of S as output; if more is needed, apply P again and read another r bits, repeating. SHA3-256 needs $256 \leq r$ bits, so one read suffices.



Each block is XORed into the rate and P is applied (*absorb*); the digest Z is then read from the rate (*squeeze*). The capacity is never injected or output.

Because the message only enters through the rate while the capacity stays hidden, the digest is *not* the whole state — so there is nothing to length-extend (§2). That is the structural advantage over Merkle–Damgård.

11.4 The permutation P (Keccak- f)

P is one fixed, unkeyed permutation of the 1600-bit state, run for 24 identical-looking *rounds*. Each round is five steps applied in sequence:

$$R = \iota \circ \chi \circ \pi \circ \rho \circ \theta, \quad P = \underbrace{R \circ R \circ \dots \circ R}_{24 \text{ rounds}}.$$

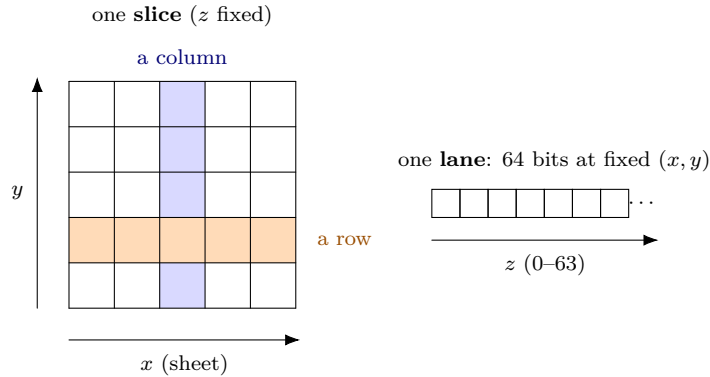
Read the composition right-to-left: θ runs first, ι last. Four of the steps (θ, ρ, π, ι) are *linear* — they only XOR and relocate bits; the fifth, χ , is the lone *nonlinear* step (it uses AND).

The state as a 3-D array. To see what the steps do, stop picturing S as a flat 1600-bit string and lay it out as a $5 \times 5 \times 64$ box of bits. Write $S[x, y, z]$ for a single bit at

$$x \in \{0, \dots, 4\} \text{ (sheet),} \quad y \in \{0, \dots, 4\} \text{ (plane),} \quad z \in \{0, \dots, 63\} \text{ (depth).}$$

Three cuts through this box have names the steps act on:

- a **lane** — the 64 bits at one (x, y) , varying z (the 25 lanes hold all $25 \times 64 = 1600$ bits); write $S[x, y]$ for a whole lane;
- a **column** — the 5 bits at one (x, z) , varying y (θ acts here);
- a **row** — the 5 bits at one (y, z) , varying x (χ acts here).



Now the five steps (all index arithmetic is mod 5 on x, y , and mod 64 on z):

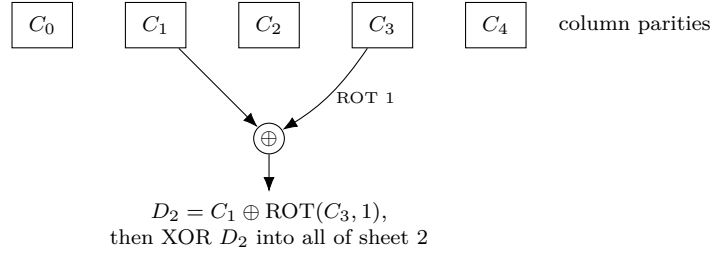
θ (**theta**) — **diffuse across columns**. Take each column's parity (XOR of its 5 bits) — one bit per (x, z) , i.e. a parity lane $C[x]$ per sheet:

$$C[x, z] = S[x, 0, z] \oplus S[x, 1, z] \oplus S[x, 2, z] \oplus S[x, 3, z] \oplus S[x, 4, z].$$

Then combine the left neighbour's parity with the right neighbour's parity rotated one step in z , and XOR that into *every* bit of sheet x :

$$D[x, z] = C[x-1, z] \oplus C[x+1, z-1], \quad S'[x, y, z] = S[x, y, z] \oplus D[x, z].$$

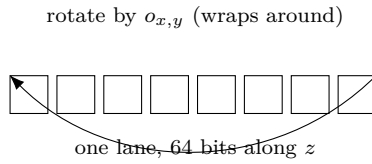
This couples every sheet to its two neighbours — the main source of diffusion.



ρ (**rho**) — **rotate each lane**. Cyclically rotate each lane along z by a fixed, per-lane offset $o_{x,y}$:

$$S'[x, y, z] = S[x, y, (z - o_{x,y}) \bmod 64].$$

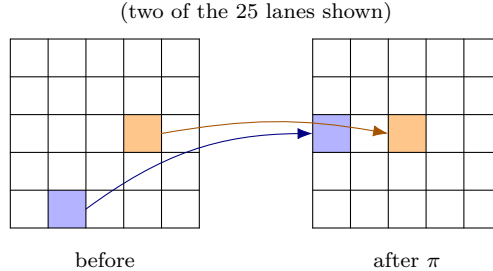
The 25 offsets $o_{x,y}$ are fixed constants ($o_{0,0} = 0$; the rest are the triangular numbers reduced mod 64). ρ is the only step that moves bits *along* a lane, so it is what mixes the 64 depth-positions.



π (**pi**) — **shuffle the lanes**. Each lane keeps its 64 bits but moves to a new (x, y) slot by a fixed rule:

$$S'[x, y] = S[(x + 3y) \bmod 5, x].$$

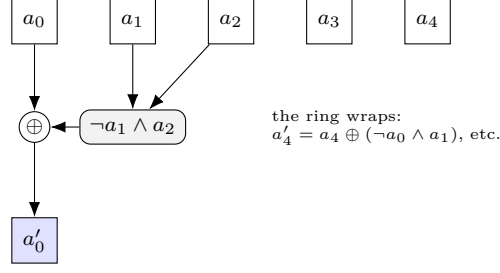
π moves no bit *inside* a lane; it regroupes the lanes so that bits θ and ρ have just spread now sit in fresh rows — the rows χ is about to mix.



χ (**chi**) — **the nonlinear mix**. Along each row (fix y, z , vary x), replace each bit using the next two bits of the row:

$$S'[x, y, z] = S[x, y, z] \oplus (\neg S[x+1, y, z] \wedge S[x+2, y, z]),$$

where $\neg$ is bitwise NOT and $\wedge$ is AND. Writing $a_x = S[x, y, z]$ for the row's five bits, this is one fixed S-box $a_x \mapsto a_x \oplus (\neg a_{x+1} \wedge a_{x+2})$ applied to every row. The AND is the *only* nonlinear operation in P ; without χ the whole permutation would be linear (pure XORs) and breakable by linear algebra.



ι (**iota**) — **break the symmetry.** XOR a round-specific constant lane RC_i into the single lane $(0, 0)$ at round i , leaving the other 24 lanes untouched:

$$S'[0, 0] = S[0, 0] \oplus RC_i \quad (i = 0, \dots, 23).$$

The RC_i differ from round to round, so the 24 rounds are not identical. Without ι every round would be the same map and a shifted input would yield a predictably shifted output — a symmetry an attacker could exploit.

Notation flag. The step π (lowercase) is one of the five steps *inside* the permutation — not the whole permutation P . Likewise $o_{x,y}$ (the ρ offsets) is unrelated to the rate r above.

11.5 Why industry still leans on SHA-2

SHA-3 is newer, has a roomy security margin, and is structurally immune to length extension (§5) — yet SHA-256 is still the default almost everywhere (TLS certificates, code signing, Git, Bitcoin, OS package managers). That is not mere laziness; four concrete reasons line up behind SHA-2.

	SHA-2 (SHA-256)	SHA-3 (SHA3-256)
Standardised	2001	2015
Structure	Merkle–Damgård	sponge
Length extension	yes (wrap in HMAC)	immune
Software speed	faster	typically slower
CPU instructions	widespread (SHA-NI, ARMv8)	rare (ARMv8.2 only)
Deployment	ubiquitous	niche / newer protocols

1. SHA-2 is not broken. The whole SHA-3 contest (2007–2012) was *insurance*: MD5 and SHA-1 had fallen (§3), both Merkle–Damgård, and the worry was that SHA-2 — also Merkle–Damgård — might be next. It was not. With no practical attack on SHA-2, there is no security reason to migrate, and NIST keeps *both* approved rather than retiring SHA-2. SHA-3’s real role is *design diversity*: a totally different construction held in reserve in case SHA-2 ever weakens.

2. Speed, especially in hardware. In plain software SHA-256 is already faster than 24-round Keccak per byte. The gap widens on real machines: modern CPUs ship *dedicated SHA-256 instructions* (Intel SHA-NI, ARMv8 crypto extensions) that hash several times faster than any portable code. Equivalent SHA-3 instructions exist only in optional, far less common silicon (ARMv8.2). So on the hardware that actually runs the internet, SHA-256 usually wins.

3. Ecosystem inertia. SHA-256 is wired into standards, file formats, and protocols deployed for two decades. Swapping a global default is expensive and risky — and buys nothing while SHA-2 holds.

4. The one wart is patchable. SHA-2’s only structural weakness, length extension, is neutralised by using HMAC (§6) or a truncated variant such as SHA-512/256. That removes the last reason one might be *forced* onto SHA-3.

Where SHA-3 does get picked: brand-new protocols that want the clean margin and no length-extension footgun, hardware where the sponge is cheap to build, and above all its XOF sibling **SHAKE** — whose variable-length output is used throughout the new post-quantum standards (ML-KEM, ML-DSA, SLH-DSA all call SHAKE internally).

11.6 Symbols

No new persistent symbols. Local:

Local symbol	Meaning (this section)
r, c	rate and capacity bits, $r + c = 1600$
P	the Keccak- f permutation (24 rounds)
S	the 1600-bit sponge state
M_i	i -th r -bit message block
$\theta, \rho, \pi, \chi, \iota$	the five steps of one round of P
x, y, z	3-D coordinates of a bit: $x, y \in \{0..4\}$, $z \in \{0..63\}$
$S[x, y, z]$, $S[x, y]$	a single state bit / a whole 64-bit lane
lane	the 64 bits at fixed (x, y) (one of the 25)
column, row	the 5 bits along y (resp. x) at fixed other coords
$C[x]$, $D[x]$	column-parity lane, and θ ’s mixing lane per sheet
$o_{x,y}$	fixed ρ rotation offset for lane (x, y)
a_x	the 5 bits of one row, $a_x = S[x, y, z]$ (used in χ)
RC_i	ι round constant for round i (0–23)
Z	squeezed output

12 Inside BLAKE3

BLAKE3 combines two ideas we have already met: a *Merkle tree* (§9) for structure, and the ChaCha ARX mixing (§3) for its compression. The result is fast, parallel, and an XOF.

12.1 Chunks as the leaves of a tree

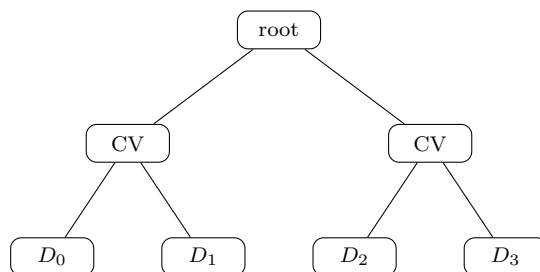
Split the input into 1 KiB *chunks* $D_0, D_1, \dots$.

Each chunk is hashed on its own into an 8-word *chaining value* (CV) — one tree leaf. Within a chunk, its (up to 16) 64-byte blocks run through f in a little chain: the CV starts at a fixed 8-word *IV* (prime-derived constants, the same as SHA-256’s; in *keyed* mode it starts from the key instead), and each block updates it, carrying a *counter* (the chunk’s index) and flags that mark the chunk’s first and last block. The chunk’s final CV is its leaf.

Adjacent leaf CVs are then combined in pairs by the *same* f , level by level, up to the *root*, whose output is the hash:

$$\text{parent CV} = f(\text{left CV} \parallel \text{right CV}).$$

A message of at most 1 KiB is a single chunk, so its CV *is* the root — no parent nodes at all.



Each 1 KiB chunk D_i compresses to a leaf chaining value; pairs of CVs combine up the tree to the root, whose squeeze is the digest.

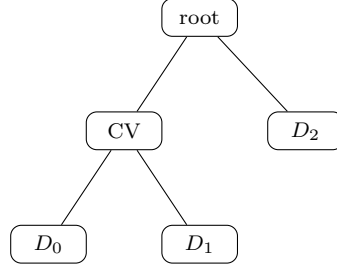
Because chunks and subtrees are independent, BLAKE3 hashes them *in parallel* — across CPU cores and SIMD lanes — which is the main reason it outruns the strictly sequential SHA-2 and SHA-3. And, exactly as in §9, you can verify a single chunk against the root with a $\log N$ authentication path, so BLAKE3 doubles as a verified-streaming primitive.

12.2 Short blocks and lopsided trees

Two ragged cases need care.

A short final block. The last 64-byte block of the last chunk is rarely full. BLAKE3 uses no bit-padding scheme (no 10*1); it zero-fills that block to 64 bytes but passes f the *true* block length as an explicit input. That length, with the counter and flags, keeps every compression’s inputs unambiguous, so distinct messages never collide through padding.

A non-power-of-two chunk count. The tree need not be perfect. BLAKE3 splits deterministically: the left subtree takes the largest power-of-two number of chunks below the total, the rest go right, and the rule recurses — nothing is duplicated (unlike the simple padding of §9). For three chunks, say:



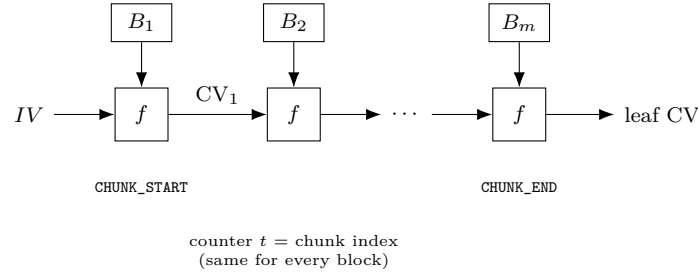
Three chunks: D_0, D_1 form the left subtree (their CVs combine into one CV); D_2 's leaf joins it at the root.

12.3 Inside a chunk: chaining the blocks

A chunk is up to 16 blocks of 64 bytes; each block is 16 words (1 word = 32 bits). The chunk is hashed by threading an 8-word chaining value through its blocks with the compression function f (opened up next):

$$CV_0 = IV, \quad CV_j = f(CV_{j-1}, B_j, t, \ell_j, d_j)_{0:8} \quad (j = 1, \dots, m).$$

Here B_j is the j -th block; t is the *counter* — the chunk's index, the *same* for every block in the chunk; ℓ_j is the block's byte length; d_j its flags; and $(\cdot)_{0:8}$ means “keep the first 8 of f 's 16 output words.” After the last block ($m \leq 16$), CV_m is the chunk's leaf. In *keyed* mode CV_0 is the key rather than IV .



The flags d_j are how BLAKE3 keeps each compression's *role* distinct — its *domain separation* (same purpose as SHA-3's domain bits, §11). The main bits:

Flag	Set when
CHUNK_START	compressing the first block of a chunk
CHUNK_END	compressing the last block of a chunk
PARENT	compressing a parent (tree) node
ROOT	the final, topmost compression

(Keyed-hash and key-derivation modes add their own flag bits.) Because the role is folded into f 's input, a leaf CV, a parent CV, and the root digest can never collide even when their other inputs happen to coincide.

12.4 The compression function f

f is the engine called everywhere above — for blocks, parents, and the root alike. It maps

$$f : \underbrace{CV}_{8 \text{ words}}, \underbrace{M}_{16 \text{ words}}, \underbrace{t}_{\text{counter}}, \underbrace{\ell}_{\text{length}}, \underbrace{d}_{\text{flags}} \mapsto 16 \text{ output words.}$$

The first 8 output words form the new chaining value; all 16 are used for root / XOF output (below). It runs in three stages.

1. Load a 4×4 word state. Lay 16 words $v_0, \dots, v_{15}$ in a grid: the 8 CV words on top, four fixed IV constants beneath, then the counter, block length, and flags.

CV ₀	CV ₁	CV ₂	CV ₃	8-word chaining value
CV ₄	CV ₅	CV ₆	CV ₇	
IV ₀	IV ₁	IV ₂	IV ₃	four IV constants
t _{lo}	t _{hi}	ℓ	d	counter, length, flags

So $v_8 \dots v_{11} = IV_0 \dots IV_3$; v_{12}, v_{13} are the low and high halves of the 64-bit counter t ; $v_{14} = \ell$; and $v_{15} = d$.

2. Seven rounds of ARX mixing. The mixing primitive is the *quarter-round* G , which stirs four state words a, b, c, d with two message words m_x, m_y :

$$\begin{aligned}
 a &\leftarrow a + b + m_x, & d &\leftarrow (d \oplus a) \ggg 16, \\
 c &\leftarrow c + d, & b &\leftarrow (b \oplus c) \ggg 12, \\
 a &\leftarrow a + b + m_y, & d &\leftarrow (d \oplus a) \ggg 8, \\
 c &\leftarrow c + d, & b &\leftarrow (b \oplus c) \ggg 7,
 \end{aligned}$$

with $+$ addition mod 2^{32} , $\oplus$ XOR, and $x \ggg k$ a right-rotation of the 32-bit word x by k bits — the Add–Rotate–XOR trio (§3). No S-boxes or lookup tables, which is what keeps it fast and constant-time. Each round applies G eight times: first down the four *columns* of the grid, then along the four *diagonals*,

columns: $G(v_0, v_4, v_8, v_{12}), \dots, G(v_3, v_7, v_{11}, v_{15})$; diagonals: $G(v_0, v_5, v_{10}, v_{15}), \dots$

a column

v ₀	v ₁	v ₂	v ₃
v ₄	v ₅	v ₆	v ₇
v ₈	v ₉	v ₁₀	v ₁₁
v ₁₂	v ₁₃	v ₁₄	v ₁₅

a diagonal

Each G consumes two message words, so all 16 are used once per round; between rounds a fixed permutation reshuffles which message word feeds which G . There are 7 rounds in total.

3. Feed-forward output. After the rounds, fold the two halves of the state together, and fold the original CV back into the lower half:

$$\text{out}_i = v_i \oplus v_{i+8}, \quad \text{out}_{i+8} = v_{i+8} \oplus \text{CV}_i \quad (i = 0, \dots, 7).$$

Re-injecting the input CV makes f *non-invertible* — you cannot run it backwards to recover the inputs — the same one-way feed-forward trick as SHA-2’s Davies–Meyer (§10). The new chaining value is $\text{out}_0 \dots \text{out}_7$.

Parents use the same f . A parent node feeds its two child CVs in as the 16-word message block, from a fresh IV , with counter 0 and the PARENT flag:

$$CV_{\text{parent}} = f(IV, CV_{\text{left}} \parallel CV_{\text{right}}, 0, 64, \text{PARENT})_{0:8}$$

(with ROOT added at the very top). One primitive, three roles — leaf, parent, root — told apart only by the flags.

12.5 Extendable output

The root can be *squeezed* for as many output bytes as you ask for (by running f on the root with an increasing counter), so BLAKE3 is natively an XOF: one function serves as a hash, a KDF (§7), and a stream generator. Default output is 256 bits.

12.6 BLAKE3 vs. SHA-3

Both are modern hashes, both are XOFs, and both are immune to length extension (§5) — yet they sit at opposite ends of a design spectrum. SHA-3 (§11) is one *sequential* permutation chosen for a conservative margin and cheap hardware; BLAKE3 is a *parallel* tree chosen for raw software speed.

	SHA-3 (SHA3-256)	BLAKE3
Core design	sponge over Keccak- f	Merkle tree of ChaCha mixes
Released	2015, NIST FIPS 202	2020, no formal standard
Parallelism	sequential (one core)	tree $\rightarrow$ multi-core + SIMD
Software speed	slower	very fast
Rounds (margin)	24 (conservative)	7 (lean, for speed)
Length extension	immune	immune
Keyed / XOF	KMAC / SHAKE	native keyed, KDF, XOF modes

Parallelism is the dividing line. SHA-3’s sponge is strictly serial: each block’s permutation needs the previous one’s output, so a single message uses one core, full stop. BLAKE3 splits the input into independent 1 KiB chunks (§12) that hash *at the same time* across cores and SIMD lanes, then combines their CVs up a tree. The same tree gives BLAKE3 $\log N$ streaming verification (§9), which a sponge cannot offer.

Speed bought with a thinner margin. On general-purpose CPUs BLAKE3 is often roughly an order of magnitude faster than SHA-3 in software. Part of that is the tree; part is doing only 7 rounds of ARX mixing where Keccak does 24. BLAKE3 is not reckless about it — its core descends from BLAKE2 and BLAKE, and BLAKE was a SHA-3 *finalist* (Keccak won) — but 7 rounds is a slimmer safety cushion than Keccak’s deliberately roomy 24, and BLAKE3 carries no government standard behind it.

Standardisation decides many real choices. SHA-3 is a NIST standard, so it is the default — sometimes the mandate — wherever compliance or a conservative long-term margin rules (government, regulated industry, protocol specs), and it is compact in dedicated hardware. BLAKE3 dominates in unregulated software where throughput matters: hashing large files, content-addressed storage, sync and deduplication, and version control. Rule of thumb: reach for SHA-3 when you need the *standard*; reach for BLAKE3 when you need the *speed* or built-in streaming verification.

12.7 Symbols

No new persistent symbols. Local:

Local symbol	Meaning (this section)
D_i	the i -th 1 KiB input chunk
B_j, ℓ_j	the j -th 64-byte block of a chunk, and its byte length
CV, CV_j	chaining value (8 words); CV_j after block j
f	BLAKE3 compression function (7 ARX rounds, feed-forward)
G	ARX quarter-round: mixes 4 state words + 2 message words
$v_0, \dots, v_{15}$	the 4×4 word state inside f
IV	fixed 8-word start of a chunk's CV (prime-derived; or the key)
t, d	f 's counter (chunk index) and flag bits (domain separation)
word	32 bits